

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 6: Organized Routing and CRUD Operations

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO3
Session	Practical 6	Date	

Aim

To separate the BookFlow API into a public catalog router (GET /books, GET /books/{book_id}) and a librarian admin router (POST, PUT, DELETE /admin/books) with APIRouter, plug both into main.py with include_router(), and validate POST/PUT bodies with Pydantic (non-negative price, ISBN as a string).

Program

bookflow_api/main.py

```
"""
Experiment 6: Organized Routing and CRUD Operations - application entry point.

Project structure:
  bookflow_api/
    main.py          entry point
    database.py      engine, session, get_db dependency
    models.py        SQLAlchemy model
    schemas.py       Pydantic models
    routes/
      catalog.py     public routes (GET /books, GET /books/{book_id})
      admin.py       librarian routes (POST, PUT, DELETE /admin/books)

Run: uvicorn main:app --reload --port 8000
"""

from contextlib import asynccontextmanager
from typing import AsyncIterator

from fastapi import FastAPI
from sqlalchemy import text

from database import engine
from routes import admin, catalog

@asynccontextmanager
async def lifespan(_: FastAPI) -> AsyncIterator[None]:
    async with engine.begin() as conn:
        await conn.execute(text("ALTER TABLE books ADD COLUMN IF NOT EXISTS price NUMERIC(10,2)"))
        await conn.execute(text(
            "ALTER TABLE books ADD COLUMN IF NOT EXISTS status VARCHAR(20) "
            "NOT NULL DEFAULT 'AVAILABLE'"
        ))
    yield
    await engine.dispose()

app = FastAPI(title="BookFlow Library API", version="2.0.0", lifespan=lifespan)

app.include_router(catalog.router)
app.include_router(admin.router)

@app.get("/", tags=["Health"])
async def root() -> dict:
    return {"service": "BookFlow Library API", "status": "running"}
```

bookflow_api/database.py

```
"""Database engine, session factory and the shared get_db dependency."""
import os
from typing import AsyncIterator

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine
```

```

from sqlalchemy.orm import DeclarativeBase

DATABASE_URL = os.getenv(
    "DATABASE_URL", "postgresql+asyncpg://postgres:postgres@localhost:5432/bookflow_db"
)

engine = create_async_engine(DATABASE_URL, pool_size=10, max_overflow=20)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

async def get_db() -> AsyncIterator[AsyncSession]:
    """Yield one async session per request and always close it afterwards."""
    async with SessionLocal() as session:
        yield session

```

bookflow_api/models.py

```

"""SQLAlchemy ORM model for the BookFlow books table."""
from decimal import Decimal
from typing import Optional

from sqlalchemy import Integer, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column

from database import Base

class Book(Base):
    __tablename__ = "books"

    book_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200), nullable=False)
    isbn: Mapped[str] = mapped_column(String(13), unique=True, nullable=False)
    published_year: Mapped[Optional[int]] = mapped_column(Integer)
    price: Mapped[Optional[Decimal]] = mapped_column(Numeric(10, 2))
    status: Mapped[str] = mapped_column(String(20), nullable=False, default="AVAILABLE")

```

bookflow_api/schemas.py

```

"""Pydantic request and response models."""
from decimal import Decimal
from enum import Enum
from typing import Optional

from pydantic import BaseModel, ConfigDict, Field

class BookStatus(str, Enum):
    AVAILABLE = "AVAILABLE"
    BORROWED = "BORROWED"
    LOST = "LOST"

class BookCreate(BaseModel):
    """Body for POST /admin/books."""
    title: str = Field(min_length=1, max_length=200)
    isbn: str = Field(min_length=10, max_length=13, description="ISBN must be a string")
    published_year: Optional[int] = Field(default=None, lt=2027)
    price: Decimal = Field(ge=0, max_digits=10, decimal_places=2, description="Not negative")
    status: BookStatus = BookStatus.AVAILABLE

class BookUpdate(BaseModel):
    """Body for PUT /admin/books/{book_id}. Only the supplied fields are changed."""
    title: Optional[str] = Field(default=None, min_length=1, max_length=200)
    isbn: Optional[str] = Field(default=None, min_length=10, max_length=13)
    published_year: Optional[int] = Field(default=None, lt=2027)
    price: Optional[Decimal] = Field(default=None, ge=0, max_digits=10, decimal_places=2)
    status: Optional[BookStatus] = None

class BookOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    book_id: int
    title: str

```

```

isbn: str
published_year: Optional[int] = None
price: Optional[Decimal] = None
status: str

```

bookflow_api/routes/catalog.py

```

"""Public catalog routes (read-only operations used by students)."""
from typing import List, Optional

from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from database import get_db
from models import Book
from schemas import BookOut

router = APIRouter(prefix="/books", tags=["Public Catalog"])

@router.get("", response_model=List[BookOut])
async def list_books(
    search: Optional[str] = Query(default=None, description="Filter by title (query parameter)"),
    skip: int = Query(default=0, ge=0),
    limit: int = Query(default=20, ge=1, le=100),
    db: AsyncSession = Depends(get_db),
) -> List[Book]:
    """GET /books - return all books, with optional title filter and pagination."""
    statement = select(Book).order_by(Book.book_id).offset(skip).limit(limit)
    if search:
        statement = statement.where(Book.title.ilike(f"%{search}%"))
    return list((await db.scalars(statement)).all())

@router.get("/{book_id}", response_model=BookOut)
async def get_book(book_id: int, db: AsyncSession = Depends(get_db)) -> Book:
    """GET /books/{book_id} - return one book (book_id is a path parameter)."""
    book = await db.get(Book, book_id)
    if book is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, f"Book {book_id} not found")
    return book

```

bookflow_api/routes/admin.py

```

"""Librarian admin routes (write operations used by staff)."""
from fastapi import APIRouter, Depends, HTTPException, Response, status
from sqlalchemy.exc import IntegrityError
from sqlalchemy.ext.asyncio import AsyncSession

from database import get_db
from models import Book
from schemas import BookCreate, BookOut, BookUpdate

router = APIRouter(prefix="/admin/books", tags=["Librarian Admin"])

async def _get_or_404(db: AsyncSession, book_id: int) -> Book:
    book = await db.get(Book, book_id)
    if book is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, f"Book {book_id} not found")
    return book

async def _commit(db: AsyncSession) -> None:
    try:
        await db.commit()
    except IntegrityError:
        await db.rollback()
        raise HTTPException(status.HTTP_409_CONFLICT, "A book with this ISBN already exists")

@router.post("", response_model=BookOut, status_code=status.HTTP_201_CREATED)
async def add_book(payload: BookCreate, db: AsyncSession = Depends(get_db)) -> Book:
    """POST /admin/books - create a new book."""
    book = Book(**payload.model_dump())
    db.add(book)
    await _commit(db)
    await db.refresh(book)
    return book

```

```
@router.put("/{book_id}", response_model=BookOut)
async def update_book(
    book_id: int, payload: BookUpdate, db: AsyncSession = Depends(get_db)
) -> Book:
    """PUT /admin/books/{book_id} - update book details such as price or status."""
    book = await _get_or_404(db, book_id)
    for field, value in payload.model_dump(exclude_unset=True).items():
        setattr(book, field, value)
    await _commit(db)
    await db.refresh(book)
    return book

@router.delete("/{book_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_book(book_id: int, db: AsyncSession = Depends(get_db)) -> Response:
    """DELETE /admin/books/{book_id} - remove a book from the system."""
    book = await _get_or_404(db, book_id)
    await db.delete(book)
    await db.commit()
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

test_p6.sh

```
#!/bin/bash
# Experiment 6 - route tests (run after: uvicorn main:app --port 8000)
BASE=http://127.0.0.1:8000
call() {
    echo "\$ curl \$*"
    body=$(curl -s -w '\n%{http_code}' "$@" )
    code=$(echo "$body" | tail -n1); json=$(echo "$body" | sed '$d')
    echo "HTTP $code"
    [ -n "$json" ] && echo "$json" | python3 -c 'import sys,json;d=json.load(sys.stdin);print(json.dumps(d,indent=2) if not isinstance(d,list) else "[\n"+",\n".join(" "+json.dumps(x) for x in d)+"\n]")'
    echo
}
echo "### Registered routes (from the OpenAPI schema)"
python3 list_routes.py
echo
echo "### 1. Public: GET /books (first 5)"
call "$BASE/books?limit=5"
echo "### 2. Public: GET /books/{book_id}"
call $BASE/books/3
echo "### 3. Public: GET /books/{book_id} for a missing book"
call $BASE/books/999
echo "### 4. Admin: POST /admin/books"
call -X POST $BASE/admin/books -H 'Content-Type: application/json' \
-d '{"title":"Fluent Python","isbn":"9781492056355","published_year":2022,"price":2450.00}'
NEW_ID=$(curl -s "$BASE/books?search=Fluent" | python3 -c 'import sys,json;print(json.load(sys.stdin)[0]["book_id"])')
echo "### 5. Admin: POST with a negative price and a numeric ISBN (rejected by Pydantic)"
call -X POST $BASE/admin/books -H 'Content-Type: application/json' \
-d '{"title":"Bad Price","isbn":9781492056356,"price":-10}'
echo "### 6. Admin: PUT /admin/books/{book_id} (change price and status)"
call -X PUT $BASE/admin/books/$NEW_ID -H 'Content-Type: application/json' \
-d '{"price":1999.00,"status":"BORROWED"}'
echo "### 7. Public: GET /books/{book_id} shows the update"
call $BASE/books/$NEW_ID
echo "### 8. Admin: DELETE /admin/books/{book_id}"
call -X DELETE $BASE/admin/books/$NEW_ID
echo "### 9. Public: GET /books/{book_id} after delete"
call $BASE/books/$NEW_ID
```

Output

Project structure and server start

```
$ tree bookflow_api
bookflow_api/
|-- main.py          entry point: creates the app and calls include_router()
|-- database.py      async engine, session factory, get_db dependency
|-- models.py        SQLAlchemy Book model
|-- schemas.py       Pydantic BookCreate / BookUpdate / BookOut
`-- routes/
    |-- __init__.py
    |-- catalog.py    public routes (GET /books, GET /books/{book_id})
    `-- admin.py      librarian routes (POST, PUT, DELETE /admin/books)
```

```
$ uvicorn main:app --port 8000
INFO:      Started server process [1481]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

bash test_p6.sh

```
### Registered routes (from the OpenAPI schema)
GET      /books                [Public Catalog]
GET      /books/{book_id}    [Public Catalog]
POST     /admin/books        [Librarian Admin]
PUT      /admin/books/{book_id} [Librarian Admin]
DELETE   /admin/books/{book_id} [Librarian Admin]
GET      /                   [Health]

### 1. Public: GET /books (first 5)
$ curl http://127.0.0.1:8000/books?limit=5
HTTP 200
[
  {"book_id": 1, "title": "Clean Code", "isbn": "9780132350884", "published_year": 2008, "price": null,
   "status": "AVAILABLE"},
  {"book_id": 2, "title": "Database System Concepts", "isbn": "9780078022159", "published_year": 2019, "price":
   null, "status": "AVAILABLE"},
  {"book_id": 3, "title": "The Martian", "isbn": "9780553418026", "published_year": 2014, "price": null,
   "status": "AVAILABLE"},
  {"book_id": 7, "title": "Dune", "isbn": "9780441172719", "published_year": 2019, "price": null, "status":
   "AVAILABLE"},
  {"book_id": 8, "title": "Atomic Habits", "isbn": "9780735211292", "published_year": 2018, "price": null,
   "status": "AVAILABLE"}
]

### 2. Public: GET /books/{book_id}
$ curl http://127.0.0.1:8000/books/3
HTTP 200
{
  "book_id": 3,
  "title": "The Martian",
  "isbn": "9780553418026",
  "published_year": 2014,
  "price": null,
  "status": "AVAILABLE"
}

### 3. Public: GET /books/{book_id} for a missing book
$ curl http://127.0.0.1:8000/books/999
HTTP 404
{
  "detail": "Book 999 not found"
}

### 4. Admin: POST /admin/books
$ curl -X POST http://127.0.0.1:8000/admin/books -H Content-Type: application/json -d {"title":"Fluent
  Python","isbn":"9781492056355","published_year":2022,"price":2450.00}
HTTP 201
{
  "book_id": 17,
  "title": "Fluent Python",
  "isbn": "9781492056355",
  "published_year": 2022,
  "price": "2450.00",
  "status": "AVAILABLE"
}

### 5. Admin: POST with a negative price and a numeric ISBN (rejected by Pydantic)
$ curl -X POST http://127.0.0.1:8000/admin/books -H Content-Type: application/json -d {"title":"Bad
  Price","isbn":9781492056356,"price":-10}
HTTP 422
{
  "detail": [
    {
      "type": "string_type",
      "loc": [
        "body",
        "isbn"
      ],
      "msg": "Input should be a valid string",
      "input": 9781492056356
    }
  ],
}
```

```

    {
      "type": "greater_than_equal",
      "loc": [
        "body",
        "price"
      ],
      "msg": "Input should be greater than or equal to 0",
      "input": -10,
      "ctx": {
        "ge": 0
      }
    }
  ]
}

### 6. Admin: PUT /admin/books/{book_id} (change price and status)
$ curl -X PUT http://127.0.0.1:8000/admin/books/17 -H Content-Type: application/json -d
  {"price":1999.00,"status":"BORROWED"}
HTTP 200
{
  "book_id": 17,
  "title": "Fluent Python",
  "isbn": "9781492056355",
  "published_year": 2022,
  "price": "1999.00",
  "status": "BORROWED"
}

### 7. Public: GET /books/{book_id} shows the update
$ curl http://127.0.0.1:8000/books/17
HTTP 200
{
  "book_id": 17,
  "title": "Fluent Python",
  "isbn": "9781492056355",
  "published_year": 2022,
  "price": "1999.00",
  "status": "BORROWED"
}

### 8. Admin: DELETE /admin/books/{book_id}
$ curl -X DELETE http://127.0.0.1:8000/admin/books/17
HTTP 204

### 9. Public: GET /books/{book_id} after delete
$ curl http://127.0.0.1:8000/books/17
HTTP 404
{
  "detail": "Book 17 not found"
}

```

Result

The six routes were registered under two tags (Public Catalog and Librarian Admin). GET returned the list and a single book (404 for a missing ID), POST created a book (201), PUT updated price and status, and DELETE returned 204 after which GET returned 404. Pydantic rejected a numeric ISBN and a negative price with 422, demonstrating full CRUD with path and query parameters and a clean separation of user-facing and administrative code.